

INNER JOIN

1. Write a query to display **employee name and department name**.

```
select employees.emp_name,departments.dept_name from employees  
inner join departments on employees.emp_id = departments.dept_id;
```

2. Write a query to list **employees working on projects**.

```
select * from employees inner join projects on employees.emp_id =  
departments.p_id;
```

3. Write a query to find **employees who are assigned to a department**.

```
select * from employees inner join departments on employees.emp_id =  
departments.dept_id;
```

4. Display **employee name and project name** using INNER JOIN.

```
select emp_name,p_name from employees inner join departments on  
employees.emp_id = departments.p_id;
```

LEFT JOIN

5. Write a query to display **all employees**, including those **without a department**.

```
select * from employees left join departments on employees.emp_id =  
departments.dept_id;
```

6. Display **all employees and their projects**, including employees with **no projects**.

```
select * from employees left join projects on employees.emp_id =  
projects.p_id;
```

7. Write a query to show **employee name and department name**, even if the department is NULL.

```
select employees.emp_name,departments.dept_name from employees left
join departments on employees.emp_id = departments.dept_id;
```

RIGHT JOIN

8. Write a query to display **all departments**, even if no employees work in them.

```
select * from departments right join employees on departments.dept_id
= employees.emp_id;
```

9. Display **all projects**, including those **not assigned to any employee**.

```
select employees.emp_name from employees right join projects on
employees.emp_id = projects.p_id;
```

FULL JOIN (Using UNION)

10. Write a query to display **all employees and all departments**, including unmatched records.

```
select employees.emp_name,departments.dept_name from employees left
join departments on employees.emp_id = departments.dept\_id;
```

union

```
select employees.emp_name,departments.dept_name from employees
right join departments on employees.emp_id = departments.dept\_id;
```

11. Show **all employees and projects**, even if no relation exists.

```
select employees.emp_name,projects.p_name from employees left join
projects on employees.emp_id = projects.p_id
```

union

```
select employees.emp_name,projects.p_name from employees
right join projects on employees.emp_id = projects.p_id;
```

CROSS JOIN

12. Write a query to display **all possible combinations of employees and departments**.

```
select * from employees cross join departments;
```

13. Display **employee names and project names** using CROSS JOIN.

```
select * from employees cross join projects;
```

14. How many rows will be returned if employee has 5 rows and department has 4 rows? (Explain using CROSS JOIN)

```
select * from employees cross join departments;
```

JOIN + CONDITIONS

18. Write a query to display employees working in the **'IT' department**.

```
select * from employees inner join departments on employees.emp_id =  
departments.dept_id where department.dept_name = 'IT';
```

19. Display employees whose **project name is 'Banking System'**.

```
select * from employees inner join projects on employees.emp_id =  
projects.p_id where projects.p_name = 'Banking System';
```

20. Write a query to list employees **without any project**.

```
select * from employees left join projects on employees.emp_id =  
projects.p_id where projects.p_name is null;
```

21. What happens if you join an **INT column with a VARCHAR column**?

5 rows in set, 3 warnings (0.0008 sec)

Warning (code 1292): Truncated incorrect DOUBLE value: 'Testing'

Warning (code 1292): Truncated incorrect DOUBLE value: 'Dev'

Warning (code 1292): Truncated incorrect DOUBLE value: 'Finance'

22. Why does SQL throw an **ambiguous column error** in joins?

```
select * from employees inner join departments on id = id;
```

23. Can we join tables **without primary and foreign keys**? Write a query.

```
select * from employees inner join departments on employees.emp_id =  
departments.dept_id;
```

24. What is the difference between:

- JOIN

Joins two different tables

- JOIN ... ON

Join on used to join two or more tables

- CROSS JOIN

It joins possible combinations of two existing tables

25. Write a query and explain the **difference between INNER JOIN and LEFT JOIN** using output.

INNER JOIN: select * from employees inner join departments on
employees.emp_id = departments.dept_id;

LEFT JOIN: select * from employees left join projects on
employees.emp_id = projects.p_id;